Department of Computer Engineering COMSATS University Islamabad, Attock Campus

Lab Rubrics Form	
Lab Name	Transformations of the Time Variable for Discrete Time Signals.
Lab Number	03
Submitted by:	

Student Names	**Registration #**
Sumaira Safeer	FA22-BCE-019

Rubrics		**Marks**		
		1	**2**	**3**
PLO-5 Modern Tools Usage	Ability to execute the experiment utilizing hardware, software tools, or a combination of both.			
PLO-10 Communication Skills	Present lab activities effectively through detailed lab reports, oral examinations, and engaging presentations.			

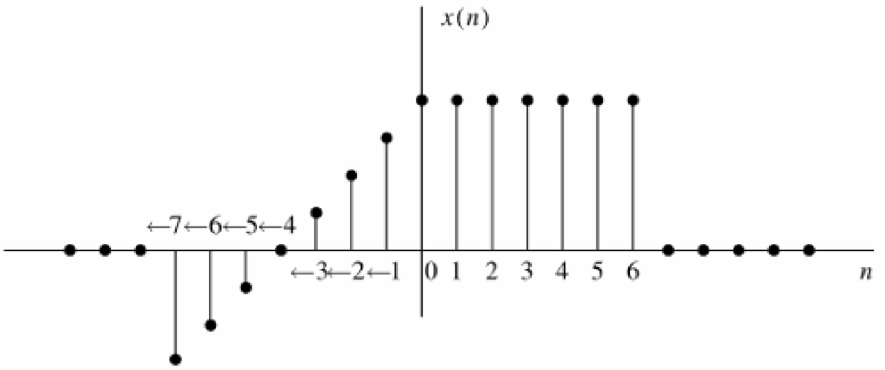
Lab-03

Transformations of the Time Variable for Discrete Time Signals

Objective:

Lab Task:

The discrete-time signal x[n] is shown in the figure below.



1. Define a discrete-time signal x[n] and its index array n.

2. Apply the following operations:

a. y1[n] =x[-n]

b. y2[n] =x[n-3]

c. y3[n] =x[n+2]

d. y4[n] =x[2n]

e. y5[n] =x[n/2]

f. y2[n] = x[-2n+4].

3. For each operation, generate the transformed signal and plot it along with the original signal using stem plots.

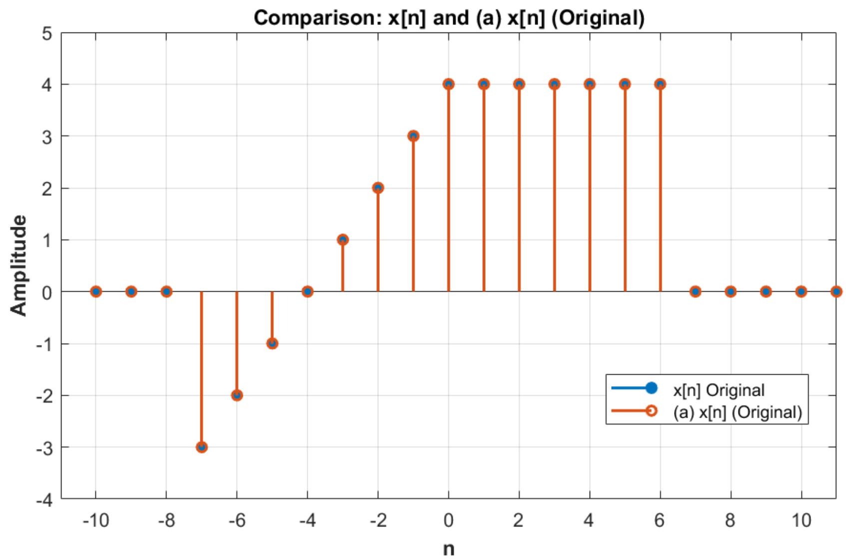
Code:

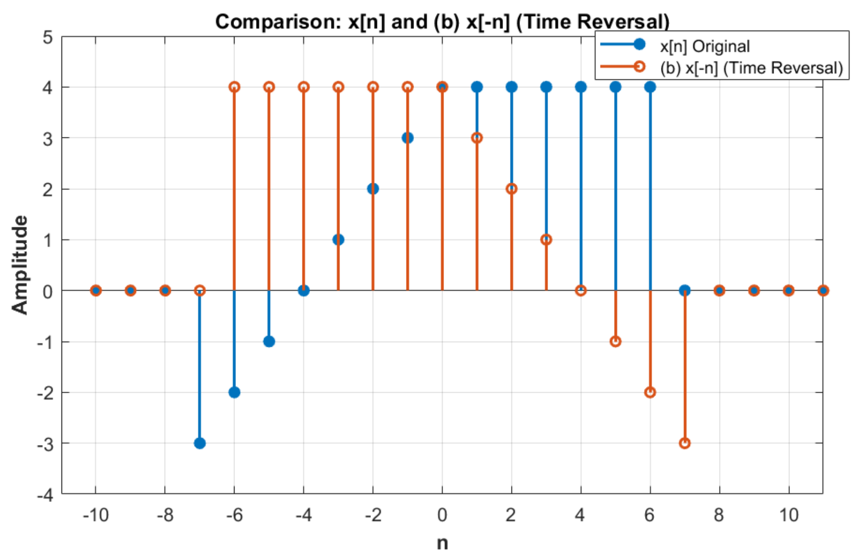
```
clc; clear; close all;
n = -10:11;
x = [0 0 0 -3 -2 -1 0 1 2 3 4 4 4 4 4 4 4 0 0 0 0 0];

N = length(n);
y = zeros(8, N);   % Store all transformed signals

% Compute transformations
for i = 1:N
    ni = n(i);
    % (a) Original
    y(1,i) = x(i);
```

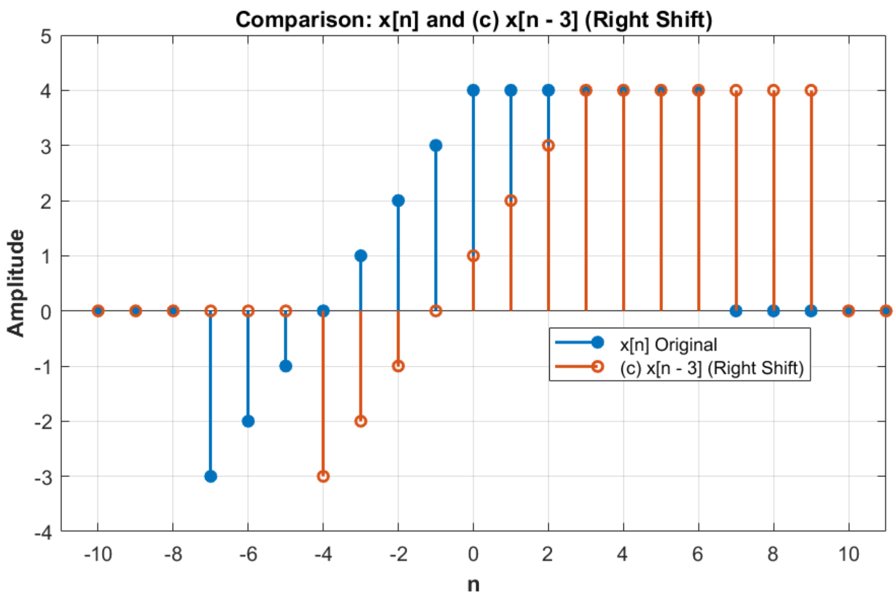
```
    % (b) Time Reversal: x[-n]
    idx = find(n == -ni, 1);
    if ~isempty(idx), y(2,i) = x(idx); end
    % (c) Right Shift: x[n - 3]
    idx = find(n == (ni - 3), 1);
    if ~isempty(idx), y(3,i) = x(idx); end
   % (d) Left Shift: x[n + 2]
    idx = find(n == (ni + 2), 1);
    if ~isempty(idx), y(4,i) = x(idx); end

    % (e) Time Compression: x[2n]
    idx = find(n == (2 * ni), 1);
    if ~isempty(idx), y(5,i) = x(idx); end

    % (f) Time Expansion: x[n/2]
    idx = find(n == (ni / 2), 1);
    if ~isempty(idx), y(6,i) = x(idx); end

    % (g) Reversal + Compression: x[-2n]
    idx = find(n == (-2 * ni), 1);
    if ~isempty(idx), y(7,i) = x(idx); end

    % (h) Reversal + Compression + Shift: x[-2n + 4]
    idx = find(n == (-2 * ni + 4), 1);
    if ~isempty(idx), y(8,i) = x(idx); end
end
% Titles for each transformation
titles = {'(a) x[n] (Original)', ...
         '(b) x[-n] (Time Reversal)', ...
         '(c) x[n - 3] (Right Shift)', ...
         '(d) x[n + 2] (Left Shift)', ...
         '(e) x[2n] (Time Compression)', ...
         '(f) x[n/2] (Time Expansion)', ...
         '(g) x[-2n] (Reversal + Compression)', ...
         '(h) x[-2n + 4] (Reversal + Compression + Shift)'};

% Plot each transformation with original signal
for k = 1:8
    figure('Name', titles{k}, 'Color', 'w', 'Position', [300 200 700 400]);

    % Plot original signal
    stem(n, x, 'filled', 'LineWidth', 1.5, 'MarkerSize', 5, 'DisplayName', 'x[n] Original');
    hold on;
    % Plot transformed signal
    stem(n, y(k,:), 'LineWidth', 1.5, 'MarkerSize', 5, 'DisplayName', titles{k});
    hold off;
    grid on;
    legend('Location', 'best');
```

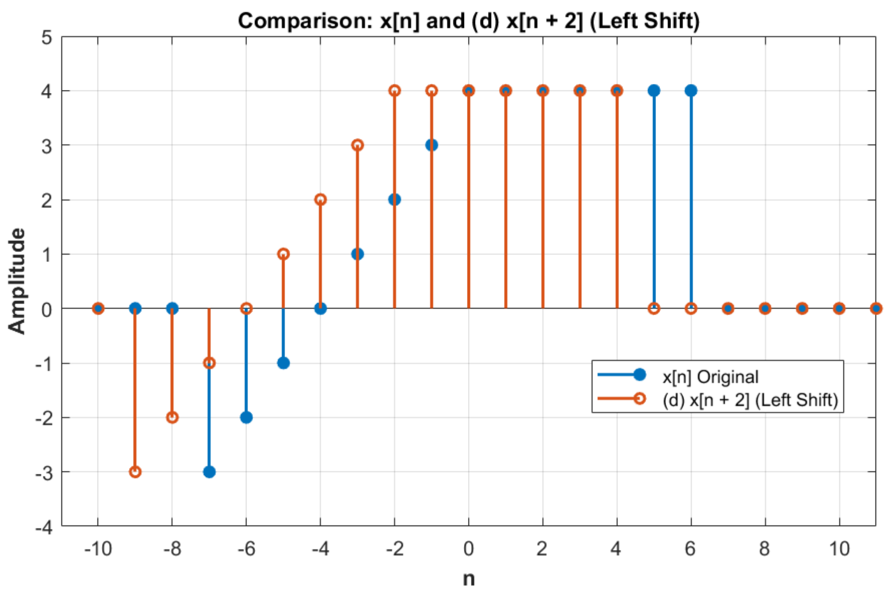
```
    title(['Comparison: x[n] and ' titles{k}], 'FontWeight', 'bold');
    xlabel('n', 'FontWeight', 'bold');
    ylabel('Amplitude', 'FontWeight', 'bold');
    xlim([-11 11]);
    ylim([min(x)-1, max(x)+1]);
end
disp('? All transformations plotted with original signal for comparison.');
```

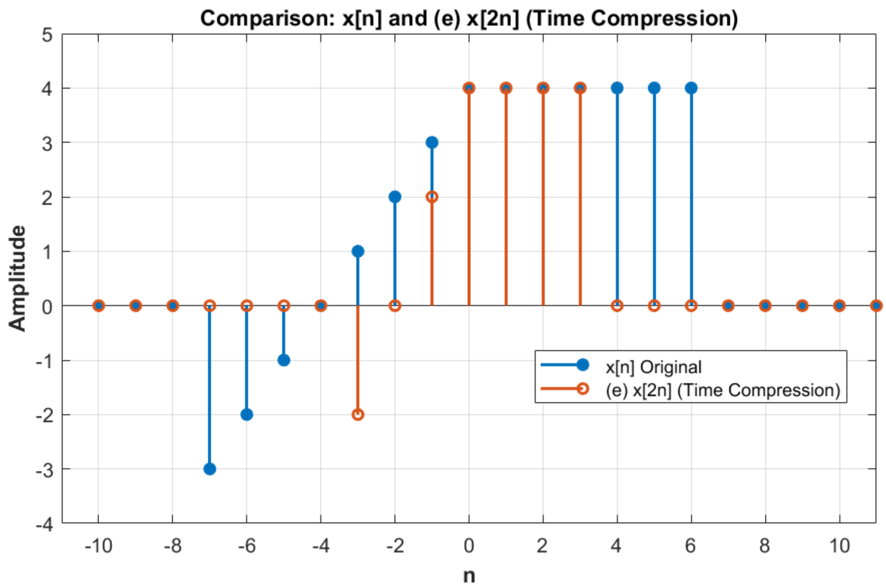
Output:

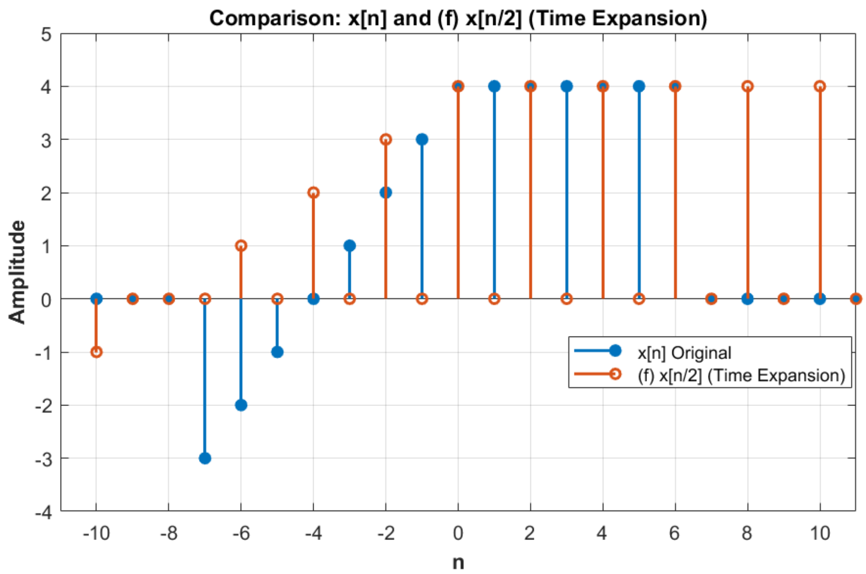


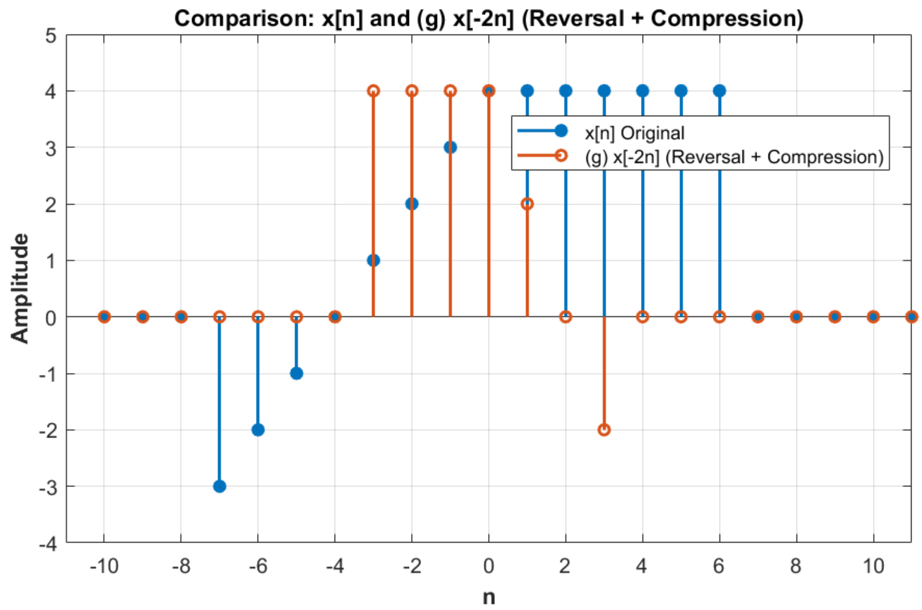


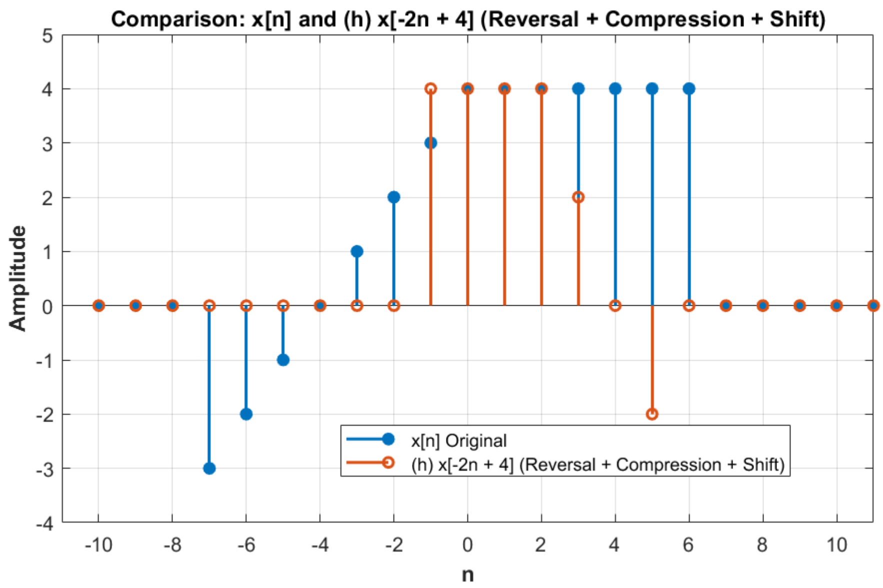

Comparison: x[n] and (c) x[n - 3] (Right Shift)
Amplitude
n
x[n] Original
(c) x[n - 3] (Right Shift)


Comparison: x[n] and (d) x[n + 2] (Left Shift)
Amplitude
n
x[n] Original
(d) x[n + 2] (Left Shift)


Comparison: x[n] and (e) x[2n] (Time Compression)
Amplitude
n
x[n] Original
(e) x[2n] (Time Compression)


Comparison: x[n] and (f) x[n/2] (Time Expansion)
Amplitude
n
x[n] Original
(f) x[n/2] (Time Expansion)


Comparison: x[n] and (g) x[-2n] (Reversal + Compression)
x[n] Original
(g) x[-2n] (Reversal + Compression)
Amplitude
n


Comparison: x[n] and (h) x[-2n + 4] (Reversal + Compression + Shift)
x[n] Original
(h) x[-2n + 4] (Reversal + Compression + Shift)
Amplitude
n